



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA INGENIERÍA EN INFORMÁTICA
CATEDRA: LENGUAJES Y COMPILADORES

ANÁLISIS LÉXICO

Profesor:
Félix Márquez

Autores:
Keibel Guilarte
C.I 28.726.602
Oriana Márquez
C.I 31.354.299
Jeanny Monagas
C.I 30.857.471
Victor Vargas
C.I 30.697.219

Ciudad Guayana, julio del 2026.

1. Defina y presente ejemplos de autómatas de pilas.

Un **Autómata de Pila (AP)** es un modelo matemático de computación que extiende la definición de un autómata finito al incorporar una estructura de datos de memoria de tipo pila con capacidad ilimitada. Esta adición permite almacenar información en un esquema LIFO (Last In, First Out), permitiendo al sistema resolver problemas de dependencia estructurada y reconocimiento de patrones que requieren memoria de anidamiento.

Formalmente, un autómata de pila se especifica mediante una tupla de siete elementos:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Donde cada componente se define teóricamente como:

- **Q:** Un conjunto finito y no vacío de estados internos.
- **Σ :** El alfabeto de entrada (un conjunto finito de símbolos permitidos en la cadena).
- **Γ :** El alfabeto de la pila (el conjunto finito de símbolos que pueden almacenarse en ella).
- **δ :** La función de transición, definida formalmente como:
$$\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$$

Donde ϵ representa la transición vacía (sin leer de la entrada) y P representa el conjunto potencia, indicando que el autómata puede realizar transiciones no deterministas.
- **q_0 :** El estado inicial de control ($q_0 \in Q$).
- **Z_0 :** El símbolo inicial de la pila ($Z_0 \in \Gamma$), que indica que la pila está vacía.
- **F:** El conjunto de estados finales o de aceptación ($F \subseteq Q$).

En la teoría de lenguajes formales, existen dos criterios matemáticos rigurosos e independientes para definir cuándo una cadena es aceptada por un autómata de pila. Estos dos enfoques son teóricamente equivalentes, ya que cualquier autómata que utilice un método puede convertirse en uno que utilice el otro:

- **Aceptación por Estado Final:** El autómata procesa por completo la cadena de entrada y finaliza su ejecución en un estado perteneciente al conjunto de estados de aceptación F . El contenido remanente en la pila es irrelevante para este criterio.
- **Aceptación por Pila Vacía:** El autómata procesa la totalidad de la cadena de

entrada y finaliza vaciando por completo todos los símbolos de la pila, incluyendo el símbolo inicial Z0. En este esquema, el estado de control final en el que se encuentre el sistema es completamente irrelevante.

A diferencia de lo que ocurre en los autómatas finitos, donde el determinismo (AFD) y el no determinismo (AFND) poseen la misma equivalencia en poder de reconocimiento de lenguajes, en los autómatas de pila el no determinismo es una propiedad matemática que incrementa la expresividad del sistema:

Criterio Teórico	Autómatas Finitos (AFD / AFND)	Autómatas de Pila (AP)
Equivalencia de Clases	$AFD \equiv AFND$. Ambos reconocen exactamente la clase de Lenguajes Regulares.	$APD \subset APND$. Los deterministas reconocen un subconjunto propio de los no deterministas.
Estructura del Lenguaje	Cualquier AFND puede convertirse a un AFD equivalente mediante la construcción de subconjuntos.	Existen lenguajes independientes del contexto no deterministas (como palíndromos de simetría central) que no pueden ser reconocidos por un AP determinista.
Límites del No Determinismo	El no determinismo solo simplifica el diseño de la máquina sin alterar su jerarquía matemática.	El no determinismo actúa como un mecanismo de predicción teórica para localizar puntos de bifurcación simétrica en las cadenas.

El poder computacional de los Autómatas de Pila se formaliza mediante su equivalencia matemática con las **Gramáticas Libres de Contexto (GLC)**, ubicadas en el Tipo 2 de la Jerarquía de Chomsky. El teorema fundamental de esta relación establece que:

Un lenguaje L es independiente del contexto si y solo si existe un autómata de pila M tal que $L = L(M)$.

Esta equivalencia teórica se fundamenta en dos demostraciones constructivas:

- **De Gramática a Autómata (GLC $\rightarrow$ AP):** Se demuestra que es posible construir un AP que simule una derivación por la izquierda de la gramática. La pila se utiliza para almacenar los símbolos no terminales activos y realizar sustituciones basadas en las reglas de producción de la gramática hasta obtener únicamente terminales.
- **De Autómata a Gramática (AP $\rightarrow$ GLC):** Se demuestra que es posible definir variables de producción gramatical que representen el viaje del autómata entre estados mientras realiza operaciones de apilado y desapilado, modelando la transición de configuración de la máquina de forma algebraica.

La relevancia teórica del Autómata de Pila radica en su posición en la jerarquía computacional. Mientras los AFD y AFND están limitados a lenguajes con patrones puramente repetitivos o finitos, los AP rompen esta barrera permitiendo modelar la sintaxis de los lenguajes de programación estructurados.

Su principal utilidad teórica es servir de base formal para el **Análisis Sintáctico (Parsing)** en la construcción de compiladores. Los algoritmos de análisis sintáctico ascendentes y descendentes actúan matemáticamente como autómatas de pila que determinan la validez jerárquica y el anidamiento estructural de las instrucciones, asegurando la correspondencia correcta de bloques lógicos antes de la generación de código.

2. Construya un lexer para la verificación de archivos docker mediante expresiones regulares.

El analizador léxico fue desarrollado desde cero en Python utilizando la librería nativa de expresiones regulares (**re**). Su propósito fundamental es realizar el análisis lineal de un archivo **Dockerfile**, agrupando los caracteres en componentes léxicos significativos (tokens) y garantizando la resiliencia del software ante errores de sintaxis en las instrucciones.

Se describe detalladamente la arquitectura y el flujo lógico implementado paso a paso:

Paso 1: Preprocesamiento y Unificación de Líneas Lógicas

La sintaxis de Docker permite romper instrucciones extensas en múltiples renglones mediante el uso del carácter de barra invertida (****). Para evitar que el lexer interprete estas rupturas como instrucciones separadas, se implementó un mecanismo de preprocesamiento utilizando la siguiente expresión regular:

```
RE_CONTINUACION = re.compile(r"\\[ \t]*\r?\n[ \t]*")
```

La función **unir_continuaciones(texto)** evalúa el archivo completo y sustituye cualquier secuencia de *backslash* seguida de un salto de línea por un único espacio en blanco. De este modo, una instrucción fragmentada físicamente se transforma en una sola línea lógica, facilitando un análisis continuo y preciso sin perder la coherencia del contexto.

Paso 2: Especificación Léxica mediante Expresiones Regulares

El núcleo del analizador se basa en una matriz de tuplas (**Nombre_Token**, **Patrón_RegEx**). La prioridad del emparejamiento está determinada estrictamente por el orden en el que se definieron las reglas dentro de la lista **TOKENS**. Los componentes clave se detallan a continuación:

- **INSTRUCTION:** Filtra las palabras clave reservadas de la gramática cerrada de Docker al inicio de una línea lógica (ej. **FROM**, **RUN**, **CMD**, **ENV**, **EXPOSE**, **HEALTHCHECK**, etc.) utilizando los límites de palabra (**\b**) y validando la coincidencia al comienzo de la línea (**^**).

- **UNKNOWN_INSTRUCTION:** Funciona como la regla de resiliencia del sistema. Si una palabra al principio de una línea lógica no coincide con ninguna instrucción válida (por ejemplo, un error sintáctico como **FROMM**), esta regla la captura para marcar explícitamente un **Error Léxico** controlado, permitiendo al lexer continuar con las siguientes líneas en lugar de colapsar la ejecución.
- **FLAG:** Identifica modificadores opcionales y banderas de configuración de comandos, soportando formatos simples o asignaciones directas (ej. **--production** o **--from=builder**).
- **KEYVALUE:** Mapea asignaciones internas de variables y metadatos estructurados en forma de par clave-valor (ej. **NODE_ENV=production** o **maintainer="Atomic Code"**).
- **IMAGE_REF:** Reconoce referencias a imágenes base del registro, soportando tanto el etiquetado por *tags* tradicionales como las referencias específicas por hashes SHA256 (ej. **node:18-alpine** o **python:3.12-slim**).
- **NUMBER:** Captura secuencias numéricas puras o aquellas asociadas a protocolos de red específicos (ej. **3000** o **8080/tcp**).
- **PUNCT e IDENTIFIER:** **PUNCT** extrae la delimitación formal de arreglos JSON ([,], ,), mientras que **IDENTIFIER** actúa como una regla comodín para capturar flujos de texto libre (argumentos de consola, rutas de archivos, operadores lógicos como **&&**, etc.) separados por espacios.

Los patrones se compilaron en un único autómata optimizado mediante el operador de alternancia (|) y el uso de grupos nombrados ((?P<nombre>patron)), procesando las cadenas en modo multilínea e ignorando la distinción entre mayúsculas y minúsculas (**re.IGNORECASE**).

Paso 3: Motor del Scanner (Generador de Tokens)

La función principal **lexer(texto)** opera como un generador eficiente en memoria (**yield**). Al recibir el flujo preprocesado, utiliza **TOKEN_REGEX.finditer(texto)** para extraer consecutivamente las coincidencias del autómata.

Durante el ciclo de escaneo, el sistema descarta los elementos puramente estéticos o de formateo como los espacios en blanco (**SKIP**) y las líneas de comentarios (**COMMENT**). Para cada token válido, el motor realiza un cálculo aritmético posicional (**mo.start()** - **inicio_linea**) para determinar con precisión

matemática la línea y la columna exacta de su ubicación antes de retornar la tupla analizada.

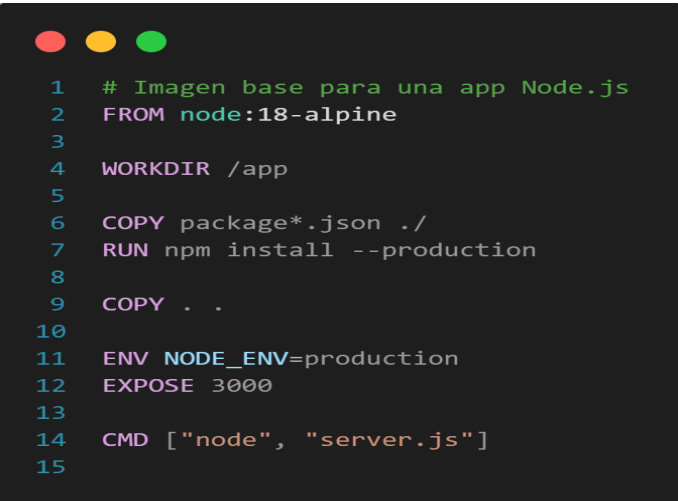
2. Ejemplos de Ejecución

Se presentan las pruebas de ejecución del analizador léxico. Cada caso demuestra la precisión en la identificación de componentes léxicos.

- **Ejemplo 1** (ejemplo1.dockerfile)

Este caso evalúa una estructura web estándar con definición de entornos, exposición de puertos y la conversión de arreglos JSON para el comando de ejecución final.

- Código de entrada



```
1  # Imagen base para una app Node.js
2  FROM node:18-alpine
3
4  WORKDIR /app
5
6  COPY package*.json ./
7  RUN npm install --production
8
9  COPY . .
10
11 ENV NODE_ENV=production
12 EXPOSE 3000
13
14 CMD ["node", "server.js"]
15
```

- Salida del Analizador Lexico

```
=====
Archivo: ejemplo1.dockerfile
=====
(INSTRUCTION, 'FROM' , línea 2, columna 0)
(IMAGE_REF , 'node:18-alpine' , línea 2, columna 5)
(INSTRUCTION, 'WORKDIR' , línea 4, columna 0)
(IDENTIFIER , '/app' , línea 4, columna 8)
(INSTRUCTION, 'COPY' , línea 6, columna 0)
(IDENTIFIER , 'package*.json' , línea 6, columna 5)
(IDENTIFIER , './' , línea 6, columna 19)
(INSTRUCTION, 'RUN' , línea 7, columna 0)
(IDENTIFIER , 'npm' , línea 7, columna 4)
(IDENTIFIER , 'install' , línea 7, columna 8)
(FLAG , '--production' , línea 7, columna 16)
(INSTRUCTION, 'COPY' , línea 9, columna 0)
(IDENTIFIER , '.' , línea 9, columna 5)
(IDENTIFIER , '.' , línea 9, columna 7)
(INSTRUCTION, 'ENV' , línea 11, columna 0)
(KEYVALUE , 'NODE_ENV=production' , línea 11, columna 4)
(INSTRUCTION, 'EXPOSE' , línea 12, columna 0)
(NUMBER , '3000' , línea 12, columna 7)
(INSTRUCTION, 'CMD' , línea 14, columna 0)
(PUNCT , '[' , línea 14, columna 4)
(STRING , '"node"' , línea 14, columna 5)
(PUNCT , ',' , línea 14, columna 11)
(STRING , '"server.js"' , línea 14, columna 13)
(PUNCT , ']' , línea 14, columna 24)
```


- Ejemplo 2 (ejemplo2.dockerfile)

Este caso de prueba demuestra el éxito del preprocesador al unificar múltiples líneas físicas conectadas por barras invertidas (\) dentro del bloque **RUN**, manteniendo de manera correcta el cálculo posicional del tokenizador. También valida la extracción de metadatos compuestos (**LABEL**).

- Código de entrada

```
1  # Imagen con argumento de build y continuacion de linea
2  ARG PYTHON_VERSION=3.12
3  FROM python:3.12-slim
4
5  LABEL maintainer="Atomic Code" \
6         version="1.0"
7
8  WORKDIR /srv/app
9
10 RUN apt-get update && \
11     apt-get install -y --no-install-recommends gcc && \
12     rm -rf /var/lib/apt/lists/*
13
14 COPY requirements.txt .
15 RUN pip install --no-cache-dir -r requirements.txt
16
17 COPY . .
18
19 USER 1000
20 EXPOSE 8080/tcp
21
22 ENTRYPOINT ["python", "app.py"]
23
```

- Salida del Analizador Lexico

```

=====
Archivo: ejemplo2.dockerfile
=====
(INSTRUCTION, 'ARG' , línea 2, columna 0)
(KEYVALUE , 'PYTHON_VERSION=3.12' , línea 2, columna 4)
(INSTRUCTION, 'FROM' , línea 3, columna 0)
(IMAGE_REF , 'python:3.12-slim' , línea 3, columna 5)
(INSTRUCTION, 'LABEL' , línea 5, columna 0)
(KEYVALUE , 'maintainer="Atomic Code"' , línea 5, columna 6)
(KEYVALUE , 'version="1.0"' , línea 5, columna 32)
(INSTRUCTION, 'WORKDIR' , línea 7, columna 0)
(IDENTIFIER , '/srv/app' , línea 7, columna 8)
(INSTRUCTION, 'RUN' , línea 9, columna 0)
(IDENTIFIER , 'apt-get' , línea 9, columna 4)
(IDENTIFIER , 'update' , línea 9, columna 12)
(IDENTIFIER , '&&' , línea 9, columna 19)
(IDENTIFIER , 'apt-get' , línea 9, columna 23)
(IDENTIFIER , 'install' , línea 9, columna 31)
(IDENTIFIER , '-y' , línea 9, columna 39)
(FLAG , '--no-install-recommends' , línea 9, columna 42)
(IDENTIFIER , 'gcc' , línea 9, columna 66)
(IDENTIFIER , '&&' , línea 9, columna 70)
(IDENTIFIER , 'rm' , línea 9, columna 74)
(IDENTIFIER , '-rf' , línea 9, columna 77)
(IDENTIFIER , '/var/lib/apt/lists/*' , línea 9, columna 81)
(INSTRUCTION, 'COPY' , línea 11, columna 0)
(IDENTIFIER , 'requirements.txt' , línea 11, columna 5)
(IDENTIFIER , '.' , línea 11, columna 22)
(INSTRUCTION, 'RUN' , línea 12, columna 0)
(IDENTIFIER , 'pip' , línea 12, columna 4)
(IDENTIFIER , 'install' , línea 12, columna 8)
(FLAG , '--no-cache-dir' , línea 12, columna 16)
(IDENTIFIER , '-r' , línea 12, columna 31)
(IDENTIFIER , 'requirements.txt' , línea 12, columna 34)
(INSTRUCTION, 'COPY' , línea 14, columna 0)
(IDENTIFIER , '.' , línea 14, columna 5)
(IDENTIFIER , '.' , línea 14, columna 7)
(INSTRUCTION, 'ENTRYPOINT' , línea 19, columna 0)
(PUNCT , '[' , línea 19, columna 11)
(STRING , '"python"' , línea 19, columna 12)
(PUNCT , ',' , línea 19, columna 20)
(STRING , '"app.py"' , línea 19, columna 22)
(PUNCT , ']' , línea 19, columna 30)

```

- Ejemplo 3: (ejemplo3.dockerfile)

Este caso de prueba valida la herencia en construcciones multi-etapa (**AS builder**), el uso de banderas con argumentos en directivas avanzadas de monitoreo (**HEALTHCHECK**) y señales de terminación del contenedor (**STOPSIGNAL**).

- Código de entrada



```
1 # Build multi-etapa para un binario en Go
2 FROM golang:1.22 AS builder
3 WORKDIR /build
4 COPY . .
5 RUN go build -o servidor .
6
7 FROM alpine:3.19
8 COPY --from=builder /build/servidor /usr/local/bin/servidor
9
10 HEALTHCHECK --interval=30s --timeout=3s CMD ["/usr/local/bin/servidor", "--health"]
11
12 STOPSIGNAL SIGTERM
13 CMD ["/usr/local/bin/servidor"]
14
```

- Salida del Analizador Lexico

```

=====
Archivo: ejemplo3.dockerfile
=====
(INSTRUCTION, 'FROM' , línea 2, columna 0)
(IMAGE_REF , 'golang:1.22' , línea 2, columna 5)
(IDENTIFIER , 'AS' , línea 2, columna 17)
(IDENTIFIER , 'builder' , línea 2, columna 20)
(INSTRUCTION, 'WORKDIR' , línea 3, columna 0)
(IDENTIFIER , '/build' , línea 3, columna 8)
(INSTRUCTION, 'COPY' , línea 4, columna 0)
(IDENTIFIER , '.' , línea 4, columna 5)
(IDENTIFIER , '.' , línea 4, columna 7)
(INSTRUCTION, 'RUN' , línea 5, columna 0)
(IDENTIFIER , 'go' , línea 5, columna 4)
(IDENTIFIER , 'build' , línea 5, columna 7)
(IDENTIFIER , '-o' , línea 5, columna 13)
(IDENTIFIER , 'servidor' , línea 5, columna 16)
(IDENTIFIER , '.' , línea 5, columna 25)
(INSTRUCTION, 'FROM' , línea 7, columna 0)
(IMAGE_REF , 'alpine:3.19' , línea 7, columna 5)
(INSTRUCTION, 'COPY' , línea 8, columna 0)
(FLAG , '--from=builder' , línea 8, columna 5)
(IDENTIFIER , '/build/servidor' , línea 8, columna 20)
(IDENTIFIER , '/usr/local/bin/servidor' , línea 8, columna 36)
(INSTRUCTION, 'HEALTHCHECK' , línea 10, columna 0)
(FLAG , '--interval=30s' , línea 10, columna 12)
(FLAG , '--timeout=3s' , línea 10, columna 27)
(IDENTIFIER , 'CMD' , línea 10, columna 40)
(PUNCT , '[' , línea 10, columna 44)
(STRING , "\"/usr/local/bin/servidor\"" , línea 10, columna 45)
(PUNCT , ',' , línea 10, columna 70)
(STRING , "\"--health\"" , línea 10, columna 72)
(PUNCT , ']' , línea 10, columna 82)
(INSTRUCTION, 'STOPSIGNAL' , línea 12, columna 0)
(IDENTIFIER , 'SIGTERM' , línea 12, columna 11)
(INSTRUCTION, 'CMD' , línea 13, columna 0)
(PUNCT , '[' , línea 13, columna 4)
(STRING , "\"/usr/local/bin/servidor\"" , línea 13, columna 5)
(PUNCT , ']' , línea 13, columna 30)

```

3. Definir un lenguaje L subconjunto de lenguaje Rust, construya un lexer para L utilizando metacompilador.

Manual de Usuario del Metacompilador (Flex)

Flex (Fast Lexical Analyzer Generator) es una herramienta de software de nivel avanzado diseñada para automatizar la creación de analizadores léxicos (scanners). En el desarrollo de compiladores, el análisis léxico es la primera fase, donde una secuencia de caracteres (código fuente) se convierte en una secuencia de tokens con significado semántico. Flex nos exime de programar manualmente los bucles y condicionales para leer carácter por carácter; en su lugar, nos permite especificar de forma declarativa los patrones de texto que conforman nuestro lenguaje utilizando **expresiones regulares**.

Al procesar un archivo de configuración (usualmente con extensión .l), Flex genera internamente las tablas de transición para un **Autómata Finito Determinístico (AFD)** altamente optimizado. El resultado de este proceso es un archivo en lenguaje C (nombrado por defecto lex.yy.c) que contiene la función principal yylex(), capaz de escanear flujos de entrada a gran velocidad.

a) Estructura del Archivo de Configuración (.l)

Todo script de Flex se divide rígidamente en tres secciones delimitadas por el símbolo %%:

- **Sección de Definiciones (arriba del primer %%):** Se utiliza para importar librerías de C (como stdio.h) dentro de los bloques %{ ... %}. También es el lugar donde se declaran las opciones del metacompilador (como %option yylineno para llevar la cuenta automática de las líneas) y se definen estados exclusivos (como %x COMENTARIO_BLOQUE). Aquí se pueden crear "alias" o macros de expresiones regulares para hacer el código más limpio.
- **Sección de Reglas (entre los dos %%):** Constituye el núcleo del analizador. Cada línea de esta sección empareja una expresión regular con un bloque de código C asociado (la acción). Cuando Flex encuentra un texto que coincide con el patrón, ejecuta el código. *Regla de oro de Flex:* Siempre coincidirá con la cadena de texto más larga posible. Si hay un empate en longitud, Flex elegirá la regla que aparezca primero en el archivo

- **Sección de Código de Usuario (debajo del último %%)**: Aquí se escribe código C puro que se copiará tal cual al final del archivo generado. Generalmente contiene la función `main()` que prepara el archivo de entrada (`yyin`) y llama a la función de análisis `yylex()`.

b) Variables Globales de Control

- **yytext**: Puntero a la cadena de caracteres (lexema) que acaba de coincidir con la regla activa.
- **yylineno**: Variable entera que almacena el número de línea actual, facilitando el reporte de errores.
- **yyin y yyout**: Punteros a archivos (flujos de entrada y salida estándar) que Flex procesará.

Descripción del Lenguaje L (Subconjunto de Rust)

El lenguaje "L" ha sido diseñado como un subconjunto directo de **Rust**, seleccionado por su sintaxis moderna y estricta. El objetivo de "L" es permitir la escritura de algoritmos fundamentales (iterativos, matemáticos y lógicos) aislando las complejidades de memoria del Rust completo, enfocándose puramente en su estructura léxica.

Categoría Léxica	Componentes / Reglas	Descripción Semántica
KEYWORD	<code>fn</code> , <code>let</code> , <code>mut</code> , <code>if</code> , <code>else</code> , <code>while</code> , <code>loop</code> , <code>return</code> , <code>break</code> , <code>continue</code> , <code>true</code> , <code>false</code>	Palabras reservadas que estructuran el control de flujo, la mutabilidad y la declaración de funciones en L.

Categoría Léxica	Componentes / Reglas	Descripción Semántica
TYPE	i32, i64, u32, u64, f32, f64, bool, char, String, str	Tipos de datos estrictos (enteros con signo/sin signo, flotantes, lógicos y alfanuméricos) característicos de Rust.
OPERATOR	+ - * / % == != < > <= >= && ! -> ::	Incluye operadores aritméticos, lógicos y específicos de Rust, como -> para denotar el tipo de retorno.
MACRO	println!	La macro de impresión se trata como un token indivisible. Flex reconoce explícitamente el sufijo de exclamación.
LITERALES	Enteros ([0-9]+), Flotantes, Cadenas ("..."), Caracteres ('.')	Valores crudos inyectados en el código. Las cadenas permiten capturar cualquier texto contenido entre comillas.
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*	Nombres definidos por el usuario para variables. Obligatoriamente deben

Categoría Léxica	Componentes / Reglas	Descripción Semántica
		iniciar con letra o guion bajo.
PUNCT	{ } () [] ; , : .	Símbolos de puntuación para agrupamiento y fin de instrucciones.

Documentación del Proceso de Creación del Lexer

El desarrollo del analizador léxico para el lenguaje "L" requirió resolver tres desafíos técnicos principales mediante el uso de expresiones regulares y directivas de estado en Flex:

a) Jerarquía y Resolución de Conflictos.

Dado que una palabra clave como `while` o `fn` cumple perfectamente con la expresión regular de un identificador genérico (empieza con letra, seguida de letras/números), se utilizó la propiedad de resolución de empates de Flex. Al colocar todas las reglas de **KEYWORD** y **TYPE** estrictamente al inicio de la sección de reglas en `lexer.l`, garantizamos que el motor del analizador asigne estos tokens específicos antes de recurrir a la categoría comodín de **IDENTIFIER**.

b) Manejo de Comentarios Multilínea mediante Estados

Los comentarios de una sola línea (`//`) se manejaron descartando todo el texto hasta el salto de línea. Sin embargo, los comentarios de bloque al estilo C (`/* ... */`) requerían una solución robusta. Se implementó un estado de arranque exclusivo (Start Condition) denominado `%x COMENTARIO_BLOQUE`. La lógica opera de la siguiente manera: al detectar los caracteres `/*`, el lexer invoca la instrucción `BEGIN(COMENTARIO_BLOQUE);`. Estando en este estado paralelo, el lexer descarta sistemáticamente cualquier carácter sin generar tokens, hasta que encuentra la

ecuencia de cierre */, momento en el cual retorna al estado normal mediante BEGIN(INITIAL);.

c) Tolerancia a Errores Léxicos (Resiliencia)

Un principio fundamental del diseño de compiladores es que un analizador no debe abortar (crashing) ante el primer error encontrado. Para cumplir esto, se programó una regla "sumidero" o "comodín" (representada por el punto .) colocada exactamente en la última línea de la sección de reglas. Si un carácter del archivo fuente (por ejemplo, @ o \$) no coincide con ninguna instrucción válida de L, caerá inevitablemente en esta última regla. En lugar de detener el programa, esta regla ejecuta un printf alertando del "ERROR LÉXICO" en la línea exacta y continúa tokenizando el resto del archivo sin interrupciones.

Guía de Instalación e Implementación (Markdown Integrado)

a) Requisitos del Sistema

- **Flex** (Fast Lexical Analyzer)
- **GCC** (Compilador de C)

b) Instalación de Dependencias

En Linux (Ubuntu/Debian):

```
sudo apt-get update  
sudo apt-get install flex gcc
```

En Windows (Vía MSYS2 / MinGW):

```
pacman -S flex gcc
```

c) Compilación del Código Fuente

Abre una terminal en el directorio del proyecto y ejecuta los siguientes dos comandos en orden. El primero genera el código en C, el segundo lo compila a un ejecutable binario.

```
Metacompile con Flex (genera lex.yy.c)  
flex -o lex.yy.c lexer.l
```

```
Compile con GCC (genera el ejecutable)  
gcc lex.yy.c -o lexer.exe
```

d) Ejecución y Pruebas

Una vez generado el ejecutable, pásale como argumento la ruta de cualquier archivo fuente en el lenguaje L (por ejemplo, los archivos de prueba ejemplo1.rs, ejemplo2.rs o ejemplo3.rs incluidos en la carpeta de ejemplos).

```
# Analizar la rutina factorial (Windows)
./lexer.exe ejemplos/ejemplo1.rs
```

```
# Analizar la rutina de números primos (Linux)
./lexer ejemplos/ejemplo2.rs
```

4. Indague y reflexione en que lenguajes en el área de seguridad informática, pudiera aplicar un FLEX, presente el lenguaje y su respectiva tokenización.

- **Lexer:** Un analizador léxico (o lexer) es un programa encargado de leer el código fuente, partirlo en piezas pequeñas y clasificarlas antes de que el compilador analice su sintaxis o significado.
- **Token vs. Lexema:** Un token representa la categoría asignada (por ejemplo, "palabra clave" o "número"), mientras que el lexema es la cadena de texto exacta y real que produjo ese token (por ejemplo, "while" o "3000").
- **Métodos de Construcción:** El informe contrasta dos enfoques para construir un lexer: desarrollarlo desde cero utilizando expresiones regulares, o generar el programa utilizando un metacompilador que recibe patrones y genera el código automáticamente.
- **Resiliencia a Errores:** Una regla teórica fundamental implementada en el proyecto es que un buen lexer no debe detenerse al encontrar el primer error. Debe reportar el problema y continuar leyendo para poder listar todos los errores del archivo en una única pasada.

a) Implementación Lexer para Dockerfile

- **Tecnología:** Este analizador fue desarrollado desde cero utilizando Python y expresiones regulares (regex) sin librerías de terceros.
- **Funcionamiento:** Analiza archivos Dockerfile identificando instrucciones válidas (como FROM, RUN, CMD, COPY). Utiliza la función `re.finditer` junto con la bandera `re.MULTILINE` para recorrer el archivo e identificar tokens, llevando la cuenta exacta de la línea y la columna de cada coincidencia.

- **Detección de Errores:** El sistema cuenta con un token llamado UNKNOWN_INSTRUCTION. Si la primera palabra de una línea no es una instrucción válida (por ejemplo, escribir "FROMM" en lugar de "FROM"), el lexer imprime el error léxico y la línea exacta, pero continúa procesando el resto del documento con normalidad.

b) Implementación Lexer para un subconjunto de Rust

- **Tecnología:** Se construyó mediante Flex, un metacompilador que lee un archivo de reglas (.l) y genera el analizador léxico en lenguaje C (lex.yy.c), el cual posteriormente se compila con gcc para crear el ejecutable.
- **Lenguaje Analizado (L):** El lexer procesa un subconjunto de Rust que incluye palabras clave (fn, let, mut, if, while), tipos de datos (i32, String, bool), operadores, puntuación y comentarios.
- **Estructura Flex:** El archivo .l contiene tres secciones: definiciones de código C puro, las reglas (patrón seguido de la acción en C) y el código principal que incluye la función main().
- **Resolución de Conflictos:** El texto explica que Flex escoge la regla que reconozca el texto más largo; en caso de empate, tiene prioridad la regla escrita primero. Por ello, las palabras clave se definen antes que los identificadores genéricos para evitar solapamientos.
- **Manejo de Comentarios:** Para ignorar los comentarios de bloque, el analizador utiliza un estado de arranque (BEGIN), permitiendo ignorar el texto interno hasta detectar el cierre del comentario.